

2211 DDP1. G DQ-office PPTMO in record block 10/09/2025.

-Prerequisite-Intro to programming

-Language C++ ^{OOP is a programming paradigm that organizes code around objects rather than functions or logic}

* Advantage of compiled language compared to interpreted & vice versa

Programming paradigm-

-Imperative-Procedural, structured, object oriented programming

-Declarative-Functional, logic programming

* Challenges of GOTO in procedural that lead to structural programming

-Program contains data and instructions

✓ Coupling is when modules are so connected that a change in one module affects the other module. ie loosely coupled is not affecting the other while tightly coupled require changes in both

✓ Cohesion is how modules are related.

-spaghetti/monolithic programming ie in C where a program would be written in main() function.

Why OOP

-Data and instructions are bundled together to avoid either data or instruction interacting with either from another bundle.

The data and instructions that are related are bundled into a pool called object. They work independently, self-contained and least interference from others. ^{easier data management for large programming}

Advantages of OOP specifically tailored for large programming

a Modular-based. This makes it easy to read and manage.

b Better maintenance since changes made in a module don't affect other modules

c Scalability. One can easily add or remove modules.

d Security. More secure since instructions or data in a module cannot access either from another module. Using principle of data hiding.

e Speed of development. ^{-Building programs from standard working modular that communicate with each other rather than code from scratch.}

f Reusability through inheritance, hence eliminate redundant code.

g Improved code organization through encapsulation

h Flexibility with polymorphism

i Simplification of complex systems through data abstraction.

Concepts of oop

- a Encapsulation - Class $\rightarrow$ object
- b Abstraction - Conceals the internal representation and manipulation of data within an object.
- c Inheritance - Ability to create new classes from existing classes
- d Polymorphism - Program exhibits different characteristics at different times

Tuesday 16th September 2025

Programming environment

- * 1. Editor $\rightarrow$ Uses simple word processor to write source code.
- * 2. Compiler $\rightarrow$ Translates source code into byte code. (object file)
- 3 Building/Linking $\rightarrow$ Combines existing file with compiled code. (.exe)
- * 4 Execution $\rightarrow$ Running of the code.

- Majorly comprises of the starred ones.

- The environments were combined to form Integrated development environment. (IDE)

*cpp - Examples of C++ IDEs: VS code, code blocks

- A project is a folder that stores files related to a program.

Syntax and conventions

- Syntax - programming rules (mandatory)

- Conventions - Agreed way of doing things i.e. good programming practices. (Not mandatory). Make program readable and easy to work with.

C++ syntax

1. Is case sensitive

2. Strongly typed language. Variables must be declared and data type stated before using the variable.

3. Uses brackets for specific purpose

() - mainly used to enclose function parameters

{ } - Create a block of statements

[] - Used with arrays.

* Pre-processor is a special instruction given to the compiler before actual code is compiled.

4 Statements end with semi-colon (;)

Conventions

1. Using descriptive names

2. Indenting

3. Commenting // or /* ---- */

Abstract datatypes-

Rules for variables

1. Must start with alphabet or underscore

2. No special character i.e. *, space, dash

3. Don't use reserved / key words i.e. int

4. Use descriptive name-convention

5. Must be unique in a given scope

6. Conventionally use camel case of first letter

C++ program building block

1. Pre-processor directives. Are of two types include or define (#)

<> - specifies location of file mostly in main library. If not in main library you can write #include "sample.h" if found in the same file or provide the whole path of file if found in external source.

Define - create constant

* All pre-processor directives must come first before writing code.

2. Namespace. You define namespace like std to prevent rewriting std::cout / cin when writing it.

3. Function. Group of statements i.e. methods in oop.

return_type name (parameters)

{

Body }

4. Statements

>> - extraction operator i.e. from keyboard

<< - Insertion operator i.e. to screen

" " - Delimiters. Text that is delimited is called literal string

Example

#include <iostream>

using namespace std;

int main () {

cout << "Hello world" << endl;

return 0;

}

cout << "Hello world \n";

Variables

- Memory locations for holding some value.

- Variables must be declared before use. i.e.

datatype identifier; int num;

Data types

1. Determines memory size allocated.
2. Size of number range that can be stored there. (determined)
3. Determines the operation and precision i.e. division of int will lead to int e.g. $\text{int } x=9, \text{int } y=4, z=x/y, \text{ans}=2.$

```
#include <iostream>
using namespace std;
int main() {
    int yob, year, age;
    printf - cout << "Enter yob\n"; "you are" << age >> "years old\n";
    scanf - cin >> yob;
    cout << "Enter current year\n";
    cin >> year;
    age = year - yob;
    cout << "You are " << age << " years old\n";
    return 0;
}
```

= Assignment
== equal to - comparison

IDE $\rightarrow$ editor i.e. notepad

Compiler

Execution

int age; int - datatype & age is identifier

Tuesday 18th November 2020

Operations

1. Arithmetic i.e. addition, subtraction, $\times$, $\div$, modulus (%).
2. Relational i.e. $>$, $>=$, $<$, $<=$, $=$, $!=$
3. Logical i.e. AND($\&\&$), OR($||$), NOT(!)
4. Increment i.e. $++$
5. Decrement i.e. $--$

Tuesday 25th November 2020

Control structures

Broadly falls into two categories,

a) Selection/Branches

b) Iteration/loops/repetition

a) Selection

- Divided into IF statement and SWITCH statement.

If statement

The versions include;

* If (condition) Statement * If... else * If... else if... else * Nested if

num += 5

Switch statement

switch (variable) {

case 1: statement

break;

case 2: statement

;

default: statement-

switch (option)

case 'a': case 'a'.

case 'A':

Statement

switch (menu)

case 1:

break;

case

- We use break so as to prevent the other cases from being executed in case the case above it is true.

```
while (number <= 10) {  
    cout << number << endl;  
    number++;  
}
```

b) Iterations/loops/repetition

- They include while, for

while (condition) {

Statements

}

for (initial; condition; update) {

Statement

}

```
for (num = 1; num <= 10; num++) {  
    cout << num << endl;  
}
```

Example

```
int actual pin = 1234;
```

```
int pin, att;
```

```
for (att = 1; att <= 3; att++) {
```



```

cout << "Enter pin /n";
cin >> pin;
if (pin == actual pin) {
    cout << "correct /n";
    break; }
else {
    cout << "Wrong pin /n";
    cout << 3 - att << "attempts left /n";
}
if (att == 3)
    cout << "Blocked /n";

```

Tuesday 2nd December 2020

Functions/modular programming

- ✓ Function is a unit performing a specific task. Each function is independent.
- Function is the smallest module of a program.

Advantages of modular programming.

- a Scalability. The ability to increase or decrease the code. Adding or removing modules in a program.
- b Readability. One can easily go through particular parts of the program unlike 'spaghetti programming'.
- c Easy to debug.
- d Reusability of code through inheritance.
- e Maintainability.

Types of methods

- a library / inbuilt functions - Part of the language. e.g. sqrt()
- b User-defined. The programmer creates these functions.
 - Function declaration e.g. void sample(^{int x}int);
 - Function definition
 - Function call

Parameters (type id, type id);
It isn't compulsory to give identifier during declaration but it is must during definition.

a. Function declaration; return type function_name(parameters); // header

b. Function definition; type name(parameters) {
Statements } // Header and body.

eg void sample(int x) {
 cout << "The argument is" << x << endl;
 cout << x << "squared is" << x * x << endl; }.

Argument is value that will be held by the parameter, which is a variable. Parameter - A variable defined in a function declaration that acts as a placeholder for the data the function needs to operate.

c. Function call; function_name(arguments);

eg sample(5);

Example int demo(int, float) {
int demo(int k, float m) { float demo(int k, float m)
float product = k * m;
cout << "The product of" << k << "and" << m << "is" << product << endl;
return 2; } return product;
int demo2(int a, int b) { float x, y; ... } }

Function call

float y = demo(3, 6); // Pass by value.

Parameter profile

1. Number of parameters.
2. Data types of the parameter.
3. Order of data types.

Passing by value/reference.

void demo(int x) {
 cout << "Original value is" << x << endl;
 x = x * 2;

define & call

*local and global variables
*Sorting an array in c++

```
cout << "The value is now" << x << endl;
```

```
}
```

```
int main() {
```

```
int number;
```

```
cout << "Enter a number \n";
```

```
cin >> number;
```

```
demo(number);
```

```
cout << "The number is" << number << endl;
```

```
return 0;
```

```
} // This is an example of pass by value.
```

- In pass by value function is given a copy of the original value and hence no changes in original.
- In pass by reference we modify the 1st line e.g. `void demo(int &x) {` i.e. the function is given the address of the variable hence changes will change the original value.

Wednesday 3rd December 2025

if (x > y) { a A program that accepts 3 numbers and determines the largest of them all.

i) Function for reading

ii) Function for checking and main

```
void largest(int x, int y, int z) {  
    int max = x;  
    if (y > max)  
        max = y;  
    if (z > max)  
        max = z;  
    cout << "largest is  
    }  
}
```

Find out on how to let user enter length of array.
b Read 10 numbers. Display as entered. Sort in ascending.
The user enters length of the array.

c Menu function square, triangle, inverted triangle, exit.
User enter pattern and number

Introduction to oop.

In every oop program, two concepts of oop must apply; encapsulation and abstraction. look at the nouns to identify classes

- Encapsulation - enclosing data and instructions into a single unit known as a class.

- Abstraction - Hiding implementation details and only showing what need to be known.

- Basic abstraction is by use of Access modifiers, in order to determine accessibility of data and methods.

a Private - A private member can only be accessed by members of the same class. By principle data should be private. e.g class students {

Private:

Class → A blueprint for creating objects.

String regNo;

}

b Protected accessibility - Accessible to members of that class and children classes.

c Public accessibility - Accessible by members and non-members of that class.

objects are created from class

Class members

- There are two types of members;

a) Data member - Refers to attributes/characteristics that defines members of a class. Define data member and the access modifier.

b) Member function (method) - Defines the functionalities of the class. Used to manipulate data members.

Class student {

Private:

String name;

float feeBalance;

Public:

Void setName();

float checkBalance();

void payFees();

scope resolution operator

If the methods are declared and have a body it is called class definition and implementation i.e. `void setName() { ... }`
`void setName();` above.

Class implementation

- ✓ It is providing details of what methods of a class do.
- ✓ Can be done in 2 ways.

a) Inline implementation `float balance() { return balance; }`

Having the body of the method within the class.

Majorly done on short methods i.e. `float balance() { return balance; }`

b) External implementation

Method is implemented outside the class.

→ `Classscope resolution operator (::)` → connects method with the class.

NB You cannot implement what you have not defined.

external implementation

```
void student::setName() {  
    cout << "Enter name of the student in";  
    cin >> name; }  
}
```

Objects

Difference between `float s;` and `Student s;`

- * Is an instance of a class.
- It assigns specific values to the data members defined in a class.
- We create objects from classes.

```
int main() {
```

```
    Student s1; // s1 is an object of type student.
```

```
    s1.setName(); } // dot syntax is object.method();
```

10/12/2025

A user is interested in having a program in which they provide radius of a circle and the program should be able to compute area and circumference.

Value that is computed isn't considered to be a stored value

The class is circle.

Data to be stored is radius

Member functions; set_radius(), get_radius(), ^{accessor} computeArea(), computeCircumference()

Access radius to compute area

Instantiation - Creating object from a class but isn't modifying the radius

Example 2

A bank requires a system to manage its customers. A customer must have a bank account in which, while opening the account. The customer provides ID#, name and amount to open account with. Once the account is opened, the customer can deposit, withdraw and check balance. Using the scenario come up with a program that suits the bank requirement.

Tuesday 16th December 2023

Program modelling

- ✓ Process of walkthrough to get the program structure and members.
- We use unified modelling language (UML)

Class diagram;

Parameters	ClassName	→	Circle
	Data members Identifier :Type		'Radius':int SetRadius():void getRadius():void getArea():void getCircumference():void
	Methods Identifier (P):Type		

-Modelling gives the mental picture of the program.

Class members

1. Data members

Variable types in OOP

1. Local variables.

- These are variables declared within a function or a block.

2. Instance variable.

- These are variables declared within a class but outside a function.

- Every object will have its own copy hence the name instance variable.
 - Exists multiple instances

- Referenced through an object.

3. Class variables

- Similar to instance variable except you use the keyword static in the declaration. e.g. students share name of university.

- Common to all objects.

- Referenced using the class.

Have a
class
scope

Types of methods

- Broadly fall into 2 categories;

a) Setters (mutators)

- These are methods that access and modify data.

b) Getters (Accessors)

- They only access but don't modify the data.

c) Constructors

- Is a special method with the following characteristics;

Constructors.

- Has the following characteristics;

1) Same name as the class.

2) Has no return type e.g. circle() is xyz :: xyz() {

3) It is invoked/called automatically once an object is created.

4) It is created automatically if the user doesn't create one. A default constructor is created.

A constructor is a special method.

- A constructor is meant to initialize the data members.

Person
name: string
gender: String
Age: int
Person():
test(int job): int
set details(): void

getAge(int job): void

Types of methods continuation

d) Destructor - special method.

- Is a special method with similar characteristics with constructor expect that its name starts with (~) i.e. tilde.
- The destructor frees or deallocates the memory.
- Destructor is automatically called at the end of the program.

e) Instance method

- This method has several instances where every object has its own copy.

f) Class method.

- Similar to instance method but has the keyword static. Belongs to the class, hence not called with an object but the class e.g. Student.setCampus()

* Constructor is called once if called more than once new objects are created. Use setter methods to create mutable objects.

Immutable object

- Is one that does not change its values after creation.
- We should create mutable instead of immutable objects.

Class circle {

Private:

int radius;

Public:

circle();


```

    void getRadius() { return radius; }
    void getArea() { cout << "Area is" << 3.142 * radius * radius << endl; }
    void setRadius();
}

circle::circle() { // constructor
    cout << "Enter radius\n";
    cin >> radius;

int main ()
circle c1; // instantiates and calls constructor
c1.getArea();
cout << "circle radius is" << c1.getRadius() << endl;
return 0;
}

```

The code above will create an immutable object. Add a setter to avoid that;

```

void circle::setRadius() {
    cout << "Enter radius\n";
    cin >> radius;
}

```